

October 18, 2025

How to Read This Document

- Each subsection describes one gameplay element and how it exists in code. So I would just look at one section and then read the code class.
- Focus on core classes in `io.github.cantstop.model`, `io.github.cantstop.rules`, and `io.github.cantstop.utensils`.
- The backend is array-only: no objects for columns, just plain arrays.

1 Gameplay Elements and Their Code Representation

1.1 Board and Columns

In the game: the board has 11 columns representing sums 2..12. Each column has a maximum height (forming a pyramid).

In code:

- Column mapping: `ConstantsBE.sumToColumnID()` and `colToSum()`.
- Heights: `ConstantsBE.maxHeight(sum)()` and `NUM_COLS = 11`.
- The state lives in `GameState`, stored in 4 parallel arrays (index 0..10):
 - `permRed[col]`, `permBlue[col]` — permanent progress of each player.
 - `temp[col]` — temporary runners during the current turn (`TEMP_NONE = -1` means none).
 - `statusArr[col]` — status codes: 0 = open, 1 = locked by RED, 2 = locked by BLUE.

Example usage:

```
1 GameState s = GameState.initial(Player.RED);
2 int col = ConstantsBE.sumToColumnID(7);
3 int max = ConstantsBE.maxHeight(7);
4
5 int red = s.redPermAtCol(col);    // permanent progress of RED
6 int temp = s.tempAtCol(col);      // temp runner (-1 = none)
7 int stat = s.statusAtCol(col);    // 0/1/2 -> open / red / blue
```

Listing 1: Reading and interpreting a column from `GameState`

Arrays instead of objects make it easier to clone, test, and integrate with AI later.

1.2 Players

In the game: two players, RED and BLUE, taking turns. After a STOP or BUST, the turn switches.

In code:

- Enum Player: RED, BLUE, with `opponent()`.
- Current player tracked by `GameState.toMove()` and `setToMove()`.
- Locked column counters: `incLocked()` and `lockedFor()`.
- Win condition: `isWin()` checks if locked columns $\geq$ `ConstantsBE.TO_WIN`.

```
1 Player current = s.toMove();
2 s.setToMove(current.opponent());
```

Listing 2: Switching player after a completed turn

1.3 Dice and Pairings

In the game: roll 4 six-sided dice, then choose one of 3 pairings (two sums). Example: (1+4, 2+5), (1+2, 4+5), (1+5, 2+4).

In code:

- `DiceRoll.roll(rng)` produces 4 random dice (1–6).
- `pairings()` returns 3 possible pairings as an array `[[a,b],[c,d],[e,f]]`.

```
1 Random rng = new Random();
2 DiceRoll roll = DiceRoll.roll(rng);
3 int[][] pairs = roll.pairings(); // 3 possible pairings
```

Listing 3: Getting 3 possible sum pairings from a roll

1.4 Move (Pairing Choice)

In the game: the player chooses one of the three pairings. Sometimes both sums point to the same column (a “double move”).

In code:

- Move: stores `pairingIndex`, `sumA`, `sumB`.
- Helper: `isSingle()` returns `true` if both sums target the same column.
- Created via `Move.of()`.

```
1 int idx = 0;
2 int a = pairs[idx][0];
3 int b = pairs[idx][1];
4 Move m = Move.of(idx, a, b);
5 if (m.isSingle()) {
6     // both sums go into the same column
7 }
```

Listing 4: Creating and inspecting a Move

1.5 Valid Moves and Bust

In the game: a move is legal only if columns are not locked and there are fewer than 3 active temporary runners. If no legal pairing exists, the player busts.

In code:

- Legal moves: `Rules.getLegalMoves(GameState, DiceRoll())`.
- Bust check: `Rules.isBust()`.
- Helper checks inside `GameState`: `isLocked()`, `isActive()`, `activeRunnersCount()`.

```
1 List<Move> legal = Rules.getLegalMoves(s, roll);
2 if (legal.isEmpty()) {
3     // bust: clear temps, switch player
4     s.clearTemps();
5     s.setToMove(s.toMove().opponent());
6 } else {
7     Move chosen = legal.get(0);
8     Rules.applyMove(s, chosen);
9 }
```

Listing 5: Detecting a bust and applying a legal move

1.6 Runners: TEMP vs PERM

In the game: temporary runners move during the turn. When the player stops, temporary progress becomes permanent.

In code:

- Add a temporary step: `GameState.addTemp(player, sum)()` (ignores locked columns and caps at `maxHeight(sum)()`).
- Commit progress: `GameState.commitTemps(player)()` (copies `temp[]` into `perm[]` and clears `temp`).
- Reset all temporary progress: `GameState.clearTemps()`.

```
1 Rules.applyMove(s, move);           // advance TEMP
2 // player decides to stop
3 s.commitTemps(s.toMove());          // commit temp -> perm
4 s.clearTemps();                     // reset temps
```

Listing 6: Typical turn fragment: TEMP advance then commit

1.7 Locking Columns and Winning

In the game: when a runner reaches the top of a column, that column becomes locked for that player. After locking three columns, that player wins.

In code:

- Inside `Rules.stop()`: after committing temps, loop over columns:
 - if `permHeight(player, col)() ≥ maxHeight(sum)()`, call `lockColumn(player, col)()` and `incLocked(player)()`.
- Win condition: `GameState.isWin(player)()` checks locked count vs `TO_WIN`.

```
1 Rules.stop(s);  
2 if (s.isWin(Player.RED)) {  
3     // RED wins the game  
4 }
```

Listing 7: Stop with automatic column lock and turn switch

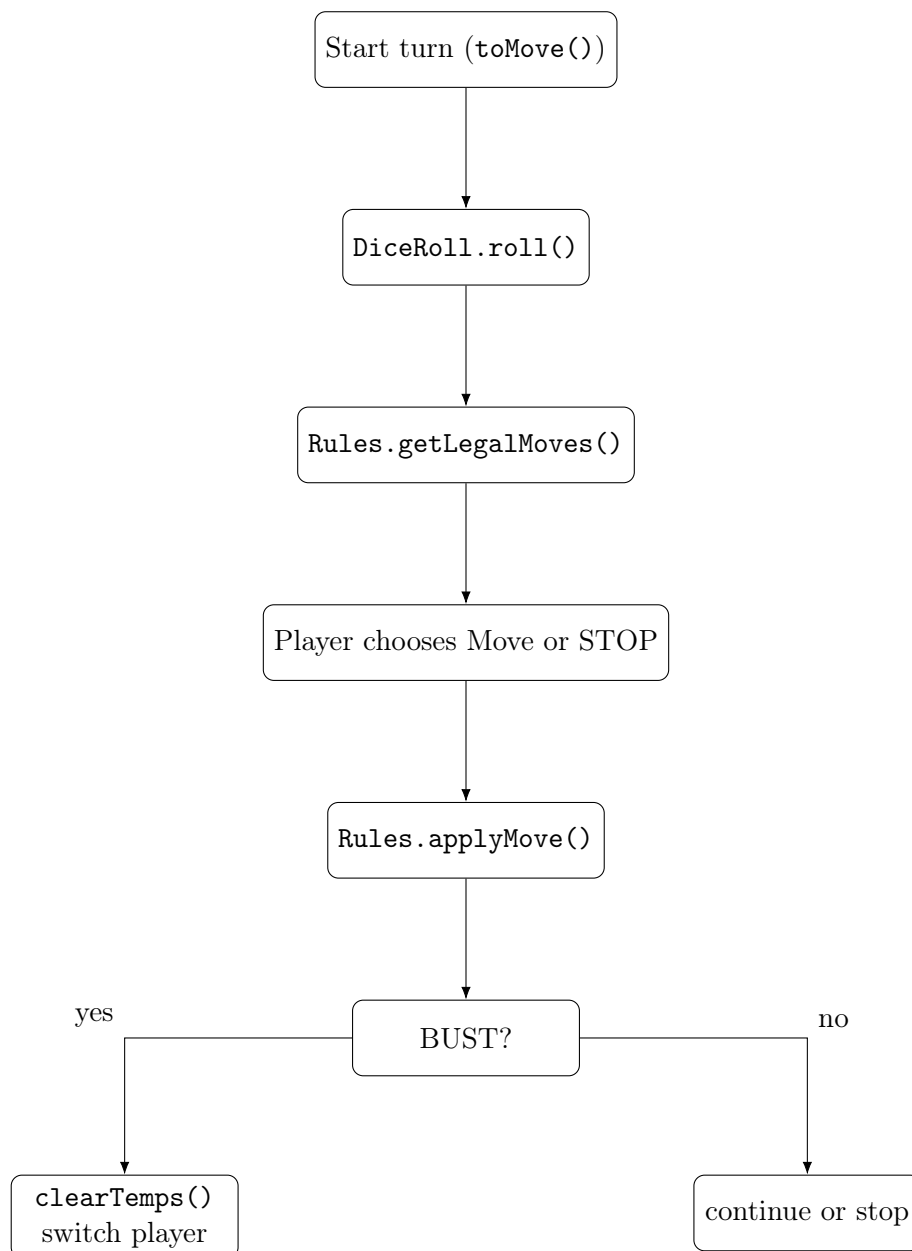
1.8 Turn Flow: CONTINUE vs STOP

In the game: after a move, the player can continue rolling (risking a bust) or stop and save progress.

In code:

- `Rules.continueTurn(s)()` – does nothing (player will roll again).
- `Rules.stop(s)()` – commits temps, locks columns, clears temps, switches player.

1.9 Turn Flow Diagram (Backend Perspective)



This flow runs every turn. The backend only enforces rules; the decision (STOP or CONTINUE) is made externally (GUI or AI).